Automated Daily Newspaper Financial Sentiment and Market Impact Dashboard using FinBERT

Advanced Artificial Intelligence (CSE_3139)
V SEMESTER MINI PROJECT REPORT
Submitted
to

MANIPAL ACADEMY OF HIGHER EDUCATION

School of Computer Engineering
CSE (Artificial Intelligence) – Section AI-A

Submitted by
TEJAS LALL
Reg. No. 245890320

Under the guidance of

Dr. Mahadevi S.
Assistant Professor
School of Computer Engineering
Manipal Institute of Technology, Bengaluru



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING
October 2026

ABSTRACT

Financial newspapers publish dozens of stories every day, and an investor who wants to judge the overall market mood has to read all of them. This project builds an end-to-end Artificial Intelligence pipeline that takes a complete daily e-paper (or a text or CSV file of headlines), extracts the individual news stories, classifies the sentiment of each story with FinBERT, a BERT language model adapted to financial text, and presents the result as an interactive Streamlit dashboard.

Two practical problems had to be solved before sentiment could be measured. First, newspaper pages are laid out in seven to eight columns, so ordinary PDF text extraction interleaves neighbouring stories. Second, the real test paper (Financial Express, Delhi edition, 12 October 2026, 42 pages) deliberately scrambles its text layer, so every standard PDF library returns unreadable characters. The system therefore detects scrambled pages automatically (0.3 s for 42 pages), reads them with RapidOCR (PP-OCRv5), and reconstructs the articles with a layout-analysis stage that combines font-height headline detection (threshold 1.28 × body-text height, calibrated on real measurements), a recursive XY-cut and a headline-ownership rule. Readable PDFs are handled directly through PyMuPDF text blocks.

Article text is split at sentence boundaries into chunks of at most 300 words and classified by ProsusAI/finbert using length-sorted mini-batches under torch.no_grad(). Each article receives a label, a confidence and an impact score P(positive) − P(negative). A rule-based extractor with 68 watchlist symbols and 72 company-name mappings links articles to stocks and indices such as NIFTY, SENSEX, TCS and INFY. The dashboard reports Bullish %, a net score, per-ticker signals and the most bullish and bearish headlines.

On pages 1–2 of the real newspaper the pipeline recovered 30 article chunks with 23 headlines and reported a 71% bullish mood with 81% mean model confidence; FinBERT inference took 6 s while OCR took about 18–26 s per page on CPU. OCR tuning reduced the time on the densest page from 102 s to 31 s. On a 17-headline sample file the system reported a 53% bearish mood with 93% mean confidence and detected all 17 referenced tickers. The report also discusses the limitations of the work: market impact is measured as the tone of the coverage, not as a price reaction, and the labels have not yet been compared with human annotations.

Keywords: Financial sentiment analysis, FinBERT, BERT, transformers, OCR, document layout analysis, XY-cut, Streamlit

Table of Contents

		Page No
	Abstract	**ii**
	List of Tables	**iv**
	List of Figures	**v**
Chapter 1	**INTRODUCTION**	**1**
1.1	Overview of the Project Work	1
1.2	Motivation and Background of the Study	1
1.3	Problem Statement and Relevance	2
1.4	Objectives	3
Chapter 2	**BACKGROUND THEORY / LITERATURE REVIEW**	**4**
2.1	Summary of Existing Studies and Comparative Analysis of Approaches	4
2.2	Evolution of the Technology over Time	6
2.3	Key Findings, Trends and Research Gaps	6
2.4	Conclusion	7
Chapter 3	**METHODOLOGY**	**8**
3.1	Methodology with Diagrams	8
3.2	System Requirements	12
3.3	Details of Data / Datasets	13
Chapter 4	**IMPLEMENTATION DETAILS AND RESULT ANALYSIS**	**15**
4.1	Results / Observations	15
4.2	Analysis of the Results	19
4.3	Comparison with Existing Work	20
Chapter 5	**CONCLUSION AND FUTURE SCOPE**	**22**
5.1	Conclusion	22
5.2	Suggestions for Future Directions	22
	REFERENCES	**24**

LIST OF TABLES

Table No	Table Title	Page No
2.1	Comparative analysis of existing approaches	5
3.1	Definitions of sentiment and market impact metrics	12
3.2	Hardware requirements	12
3.3	Software requirements and tested versions	13
3.4	Input formats and test data	13
4.1	Key implementation parameters	15
4.2	Results on the Financial Express e-paper	15
4.3	Ticker extraction on sample_news.txt	19
4.4	Comparison of the proposed system with existing work	20

LIST OF FIGURES

Figure No	Figure Title	Page No
2.1	Evolution of financial sentiment analysis techniques	6
3.1	System architecture of the sentiment and market impact pipeline	8
3.2	Layout analysis: (a) the problem, (b) recursive XY-cut, (c) headline ownership	10
3.3	FinBERT inference procedure and derived scores	11
4.1	Detected article layout on page 3 of the Financial Express e-paper	16
4.2	Detected article layout on page 1 (left) and page 10 (right)	17
4.3	OCR time per page and the effect of OCR tuning	18
4.4	Sentiment distribution for the newspaper run and the sample headline set	18

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project Work

News is one of the main channels through which information reaches financial markets. Earnings surprises, policy announcements, regulatory actions and corporate events are first reported by the press, and the tone of that reporting influences how investors interpret them. A business daily such as the Financial Express carries several dozen stories across forty or more pages, and reading every one of them to form a view of the day's market mood is slow and subjective.

This project, titled “Automated Daily Newspaper Financial Sentiment and Market Impact Dashboard”, automates that reading. The user uploads a whole day's newspaper as a PDF e-paper, or a text or CSV file of headlines, and within a few minutes the application shows: the dominant market mood (for example “71% Bullish”); the sentiment of every story with a confidence score; which stocks and indices are mentioned and in what tone; and the most bullish and most bearish headlines, which can be filtered and exported.

The project combines three areas of Artificial Intelligence. Natural Language Processing is provided by FinBERT [1], a BERT model [2] that has been further pre-trained on financial text and fine-tuned for three-class financial sentiment. Computer Vision is provided by optical character recognition (RapidOCR with PP-OCRv5 models [13], [23]) for pages whose text layer cannot be read. Classical document-analysis algorithms, a recursive XY-cut [12] and a headline-ownership heuristic, rebuild the reading order of multi-column newspaper pages. The complete system is a single Python application of about 2,240 lines with a Streamlit [21] user interface and Plotly [24] charts.

1.2 Motivation and Background of the Study

Research in finance has shown that the tone of news is related to market behaviour. Tetlock [6] found that high levels of media pessimism in a Wall Street Journal column predicted downward pressure on market prices, and Bollen et al. [7] reported that the collective mood of Twitter messages helped forecast movements of the Dow Jones index. Sentiment extracted from text is therefore a useful signal for traders, analysts and researchers.

Generic sentiment tools are poorly suited to financial language. Loughran and McDonald [5] showed that almost three quarters of the words that a general-purpose dictionary marks as negative, such as “tax”, “cost”, “liability” or “risk”, are not negative in a financial context. A sentence such as “the company reduced its losses” is positive for investors even though it contains a negative word. Domain-specific language models such as FinBERT address this by learning from financial text, and they currently give much higher accuracy than dictionary methods on financial sentiment benchmarks [1].

Most published systems, however, assume clean input: one sentence or one headline at a time, already separated. A real newspaper is very different. Its pages are printed in many columns, headlines span several columns, pull quotes and captions are mixed with body text, and pages contain advertisements, legal notices and stock-price tables. In addition, some e-paper publishers deliberately scramble the text layer of their PDFs to prevent copying. The motivation for this project was to build a system that works on such real input, end to end, rather than on a pre-cleaned dataset.

1.3 Problem Statement and Relevance

Problem statement: Given a complete daily financial newspaper in PDF form (or a set of headlines), automatically extract each news story, determine its financial sentiment, identify the stocks and indices it refers to, and summarise the day's market mood in an interactive dashboard, on an ordinary CPU and within minutes.

Solving this problem requires overcoming the following technical challenges:

1. Multi-column layout. A plain PDF text extractor reads each horizontal line straight across the page, so line 1 of column 1 is followed by line 1 of column 2 and neighbouring stories are interleaved. The sentiment model would then score meaningless mixtures of different stories.
2. Scrambled text layers. In the Financial Express test paper, pdfplumber, PyPDF2 and PDFium return the replacement character “�” for 100% of the characters, and PyMuPDF returns control codes such as \x01\x02\x03. Without OCR the model would be scoring garbage.
3. Article segmentation. After OCR, the system only has a list of text lines with positions. It must decide which lines are headlines and which body lines belong to which headline.

4. Model limits. BERT-based models accept at most 512 tokens, so long articles must be split without silently cutting text off, and inference on hundreds of chunks must remain fast on a CPU.
5. Entity linking. Sentiment is only useful to an investor when it is attached to a stock or index, so tickers and company names must be recognised without flagging words such as CEO, GDP or RBI.

The work is relevant to retail investors and analysts who want a quick, consistent summary of the day's news; to researchers who need sentiment time series built from newspapers rather than social media; and, more generally, as a case study of applying a pre-trained transformer to noisy real-world documents.

1.4 Objectives

The objectives of the project are:

1. To accept multiple input sources: digital and scanned/scrambled PDF e-papers, .txt headline files, .csv article datasets and single headlines typed by the user.
2. To detect automatically whether a PDF page has a readable text layer and, when it does not, to read it with OCR.
3. To reconstruct individual newspaper articles from multi-column pages using layout analysis, and to remove running headers, footers, captions and stock tables.
4. To classify each article into positive, negative or neutral sentiment with FinBERT, using efficient batched inference that never truncates article text.
5. To extract stock tickers and index names from each article and compute a per-ticker sentiment signal.
6. To define clear market-mood metrics (Bullish %, net score, impact score) and present them in an interactive, filterable dashboard with CSV export.
7. To evaluate the system on a real 42-page newspaper and on a sample headline set, and to document its limitations honestly.

CHAPTER 2
BACKGROUND THEORY / LITERATURE REVIEW

2.1 Summary of Existing Studies and Comparative Analysis of Approaches

Approaches to financial sentiment analysis fall into three broad families: lexicon-based methods, classical machine learning and deep learning, and pre-trained transformer models. Document analysis and OCR form a separate body of work that is needed whenever the input is a printed or scanned page.

2.1.1 Lexicon-based methods

Lexicon methods count words from predefined positive and negative lists. Tetlock [6] used the General Inquirer dictionary to measure pessimism in a daily Wall Street Journal column and linked it to subsequent market returns. Loughran and McDonald [5] showed that general dictionaries misclassify many financial terms and produced finance-specific word lists from 10-K filings, which became the standard lexicon in accounting and finance research. VADER [8] is a rule-based model with valence scores, intensifiers and negation handling, designed for social-media text. Lexicon methods are transparent and fast, but they ignore word order and context, so they cannot handle sentences such as “losses narrowed”.

2.1.2 Classical machine learning and deep learning

Supervised models learn the mapping from text to sentiment from labelled examples. Malo et al. [4] released the Financial PhraseBank, about 4,840 sentences from financial news labelled by 16 annotators with finance backgrounds, which became the main benchmark for the task. Early models used bag-of-words features with Naive Bayes or support vector machines. Neural models such as convolutional networks for sentence classification [9] and recurrent LSTM networks [15] with word embeddings improved results by capturing local context, but they need large labelled datasets, which are scarce in finance.

2.1.3 Pre-trained transformer models

The Transformer architecture [3] replaced recurrence with self-attention, allowing every token to attend to every other token in parallel. BERT [2] pre-trains a 12-layer Transformer encoder on large general corpora with masked-language-modelling and next-sentence objectives, after which it can be fine-tuned for classification with little labelled data.

FinBERT (Araci, 2019) [1] further pre-trains BERT on a financial corpus (TRC2-financial) and fine-tunes it on the Financial PhraseBank. Araci reports an accuracy of 0.86 and macro F1 of 0.84 on the full PhraseBank and 0.97 accuracy on the subset where all annotators agree, compared with 0.71 for an LSTM baseline and 0.83 for ULMFit. This model is published on Hugging Face as ProsusAI/finbert [20] and is the model used in this project. A separate FinBERT by Huang et al. [10] is pre-trained on analyst reports and corporate filings, and very large domain models such as BloombergGPT [11] show that the trend towards domain-specific language models continues.

2.1.4 Document layout analysis and OCR

Before text can be classified it must be extracted in the correct reading order. The recursive XY-cut algorithm of Nagy and Seth [12] splits a page into regions by repeatedly cutting along empty horizontal and vertical bands of whitespace, producing a tree whose leaves are text blocks. It remains a standard baseline for Manhattan layouts such as newspapers. For pages without a usable text layer, OCR is required; PP-OCR [13] is a practical, lightweight OCR system made of a text detector and a CRNN-style recogniser, and RapidOCR [23] packages PP-OCR models for CPU inference through ONNX Runtime.

Table 2.1 compares the main approaches with the system developed in this project.

Table 2.1: Comparative analysis of existing approaches

Approach / work	Technique	Typical input	Strengths	Limitations
Tetlock [6]	General Inquirer word counts	One newspaper column	Simple, interpretable; links tone to returns	General dictionary; no context
Loughran–McDonald [5]	Finance-specific word lists	10-K filings	Domain vocabulary; widely used	Ignores word order and negation
VADER [8]	Rule-based valence lexicon	Social-media posts	No training data needed	Not tuned for financial language
CNN / LSTM [9], [15]	Supervised neural networks	Labelled sentences	Captures local context	Needs large labelled sets
FinBERT [1]	BERT further pre-trained + fine-tuned	Single sentences	0.86 accuracy on PhraseBank	512-token limit; clean text assumed
This project	FinBERT + OCR + layout analysis + ticker rules	Whole newspaper PDF, TXT, CSV	End-to-end on real e-papers; per-ticker view	No price data; not yet evaluated on human labels

2.2 Evolution of the Technology over Time

Figure 2.1 summarises how financial sentiment analysis has evolved. The first generation (around 2007–2011) relied on dictionaries and showed that news tone carries information about markets. The second generation (around 2012–2016) introduced labelled datasets such as the Financial PhraseBank and supervised learners, first with hand-crafted features and later with word embeddings, CNNs and LSTMs. The third generation began with the Transformer in 2017 and BERT in 2018; transfer learning made it possible to reach high accuracy with only a few thousand labelled financial sentences, and FinBERT followed in 2019. Since 2020, open-source libraries such as Hugging Face Transformers [14] have made these models easy to deploy, lightweight OCR systems such as PP-OCR have made reading scanned documents practical on CPUs, and large domain-specific language models have appeared.

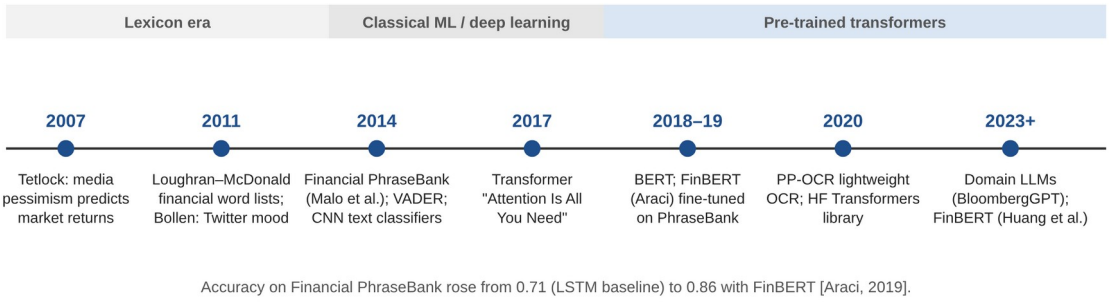


Figure 2.1: Evolution of financial sentiment analysis techniques

2.3 Key Findings, Trends and Research Gaps

The review leads to the following findings:

- Domain adaptation matters. Financial language differs from everyday language, and models trained or further pre-trained on financial text consistently outperform general ones [1], [5].
- Transfer learning reduces the need for labels. Pre-trained transformers reach high accuracy with a few thousand labelled sentences, which suits finance, where expert labels are expensive.
- News tone is informative. Several studies relate media sentiment to subsequent returns and trading volume [6], [7], which motivates aggregating sentiment at the level of a day and of a stock.

- The trend is towards larger domain models and towards end-to-end pipelines that combine document understanding with language models.

The following research gaps were identified and are addressed, at least partly, by this project:

- Benchmarks and most published systems evaluate isolated sentences; very few process an entire, real newspaper with its multi-column layout, advertisements and legal notices.
- Robustness to unreadable PDF text layers is rarely discussed, although it is common in commercial e-papers.
- Indian financial news, with its own indices (NIFTY, SENSEX, Bank Nifty) and company names, is under-represented compared with US news.
- Sentiment scores are often reported per sentence, without linking them to tickers or aggregating them into an interpretable daily market-mood indicator.

2.4 Conclusion

Pre-trained, domain-adapted transformers such as FinBERT are the current state of practice for financial sentiment classification, clearly ahead of lexicon and classical approaches. Their weakness in practice is not the classifier but the input: they assume clean, short, correctly segmented text. The literature on document layout analysis and OCR provides the missing tools. This project therefore combines FinBERT with OCR and layout analysis to apply state-of-the-art sentiment classification to real newspapers, and adds ticker linking and daily aggregation to make the output useful to an investor.

CHAPTER 3
METHODOLOGY

3.1 Methodology with Diagrams

The system is organised as a pipeline of four stages: input, text extraction and layout analysis, natural language processing, and analytics and presentation. Figure 3.1 shows the complete architecture. Each stage is described in the subsections below.

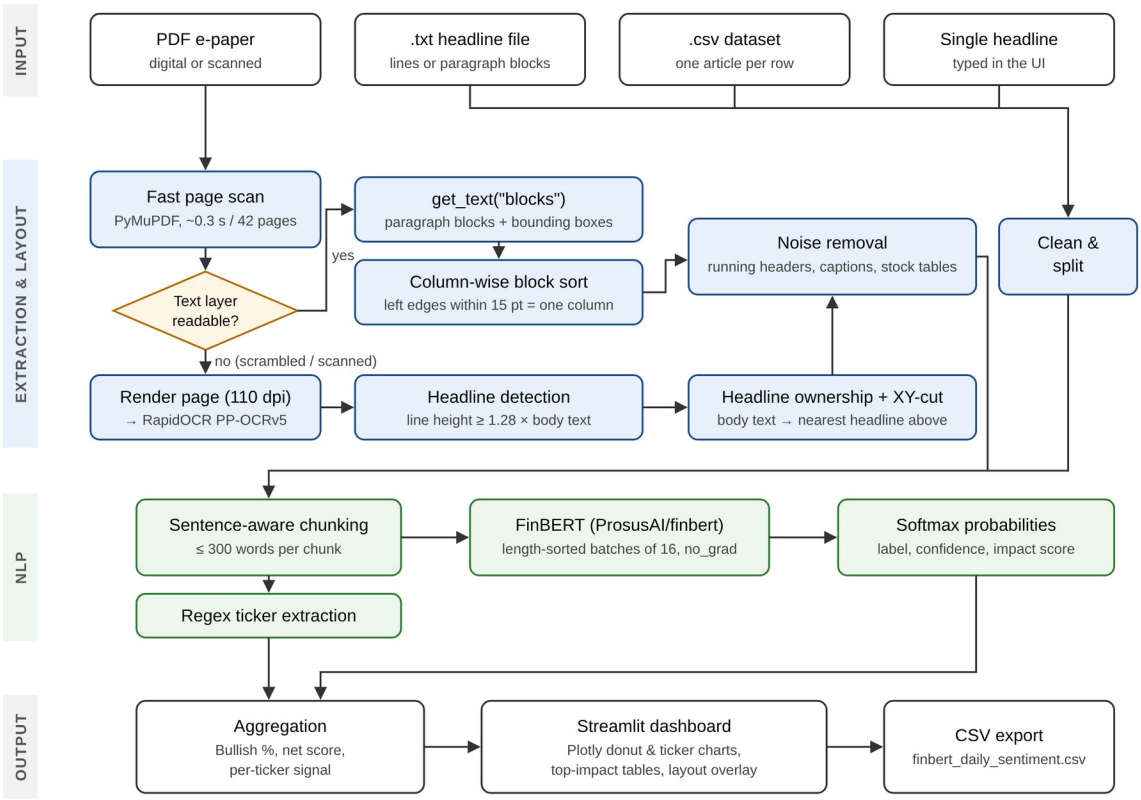


Figure 3.1: System architecture of the sentiment and market impact pipeline

3.1.1 Input handling

The application accepts five kinds of input, summarised later in Table 3.4. Text files are decoded, cleaned (URLs, e-mail addresses, bullets, dates and page furniture are removed, and words hyphenated across line breaks are re-joined) and split either one item per line or into paragraph blocks; the split mode is detected automatically from the shape of the text. For CSV files the column containing the article text is detected from its name (headline, title, text, content and similar) and can be changed by the user. PDF files go through the pipeline described next.

3.1.2 Scrambled text-layer detection

Every page of an uploaded PDF is first scanned with PyMuPDF [22]. A page is marked as unreadable (“garbled”) if it has fewer than 50 visible characters, as is the case for scanned images, or if more than 20% of its visible characters are replacement characters, private-use code points or control codes. This check requires no OCR and took about 0.3 s for all 42 pages of the test paper. Readable pages are processed from their text layer; garbled pages are rendered to an image and passed to OCR.

$$garbled(page) = (visible < 50) \ \ OR \ \ (bad\ characters / visible\ characters > 0.2)$$

3.1.3 Readable pages: column-wise block ordering

For readable pages, PyMuPDF's get_text("blocks") returns paragraph-like blocks with their bounding boxes. Blocks whose left edges lie within 15 points of each other are assigned to the same column, and the blocks are sorted column by column and top to bottom within a column. Tiny blocks (page numbers, bylines), running headers and footers that repeat in the top or bottom 7% of several pages, captions and numeric stock tables are removed before the remaining blocks are turned into article chunks.

3.1.4 Scrambled pages: OCR and layout analysis

Garbled pages are rendered at 110 dpi, at which 13.6 pt newspaper body text is about 21 pixels tall, and read with RapidOCR using the English PP-OCRv5 recogniser, a maximum image side of 6,000 pixels and recognition batches of 32 lines. OCR lines with a confidence below 0.5 are discarded and the remaining line boxes are converted back to PDF points. The body-text size of the page is estimated as the most common line height weighted by the number of characters. Lines smaller than 0.75 × or larger than 7 × the body size (photo credits, mastheads, giant section titles) are dropped, and large drop capitals are re-attached to the first line of their paragraph. Articles are then reconstructed in three steps, illustrated in Figure 3.2.

1. Headline detection. A line is a headline candidate if its height is at least 1.28 times the body-text height, or if it is a short, bold, slightly larger line that does not end with a full stop. The 1.28 threshold was calibrated on the real e-paper: pull quotes measured at most 1.23 × the body height, and the smallest real headlines at least 1.31 ×. Consecutive headline lines that touch and overlap horizontally are merged into one headline block (kicker plus multi-line headline).

2. Headline ownership. Newspapers place a headline on top of its own story, and the story runs in the columns directly beneath it. Each body line is therefore assigned to the nearest headline block above it that horizontally covers it. A headline spanning three columns owns all three columns until a lower headline takes over.
3. Reading order with recursive XY-cut. The lines owned by each headline, and any lines with no headline above them, are ordered with a recursive XY-cut. Text boxes are projected onto the y- and x-axes; a horizontal gap of at least 1.8 × body size separates stacked stories, a vertical gap that no box crosses is a column gutter, a headline that spans gutters is first cut above and below so that the columns beneath can be separated, and finally smaller paragraph gaps of at least 1 × body size are used. The recursion stops when nothing more can be cut, at which point each leaf is a single column of text.

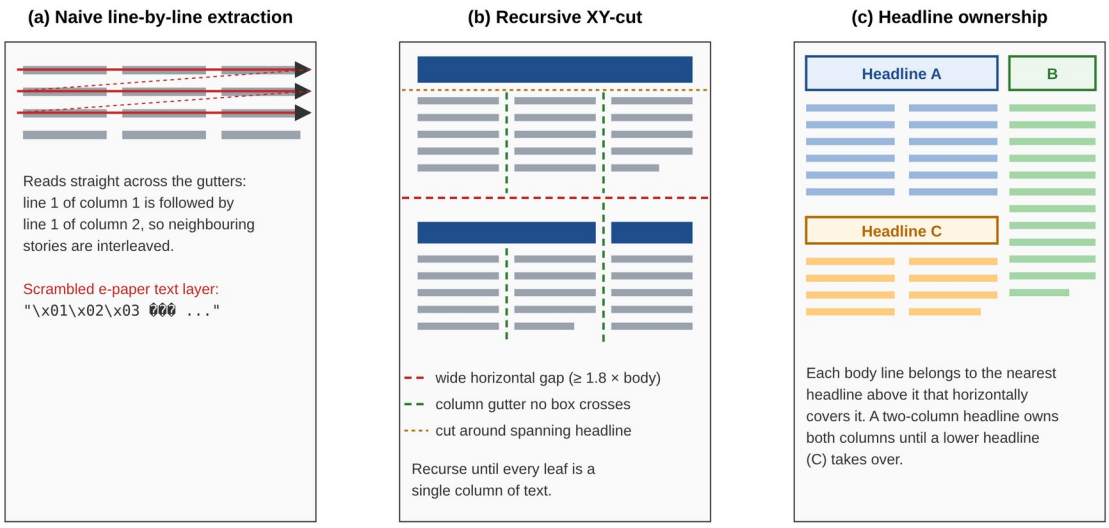


Figure 3.2: Layout analysis: (a) the problem, (b) recursive XY-cut, (c) headline ownership

3.1.5 Sentence-aware chunking

FinBERT accepts at most 512 WordPiece tokens. Rather than letting the tokenizer silently truncate long articles, the text is split at sentence boundaries into chunks of at most 300 words, which keeps each chunk safely inside the token limit. Long stories therefore appear as “part 1/2”, “part 2/2” and so on. Chunks shorter than a user-selectable minimum word count are dropped, and duplicate chunks are removed.

3.1.6 FinBERT sentiment inference

Figure 3.3 shows the inference procedure. The ProsusAI/finbert model and its tokenizer are loaded once per server with Streamlit's @st.cache_resource, because Streamlit re-runs the script on every interaction. Chunks are sorted by length and grouped into mini-batches (default 16, selectable between 4 and 64); sorting means that short headlines are not padded to the length of long articles in the same batch. Each batch is tokenised with WordPiece [16], which adds the [CLS] and [SEP] tokens, applies dynamic padding to the longest sequence in the batch, truncates at 512 tokens and builds an attention mask. The model runs in eval() mode under torch.no_grad(), which turns off gradient bookkeeping, and produces three logits per text from the [CLS] representation.

The logits z are converted to probabilities with the softmax function. The predicted class is the arg-max and its probability is reported as the model's confidence. Class names are read from model.config.id2label rather than hard-coded, and results are written back to their original order.

$$p_i = \exp(z_i) / \Sigma_j \exp(z_j), \quad label = \operatorname{argmax}_i p_i, \quad confidence = \max_i p_i$$

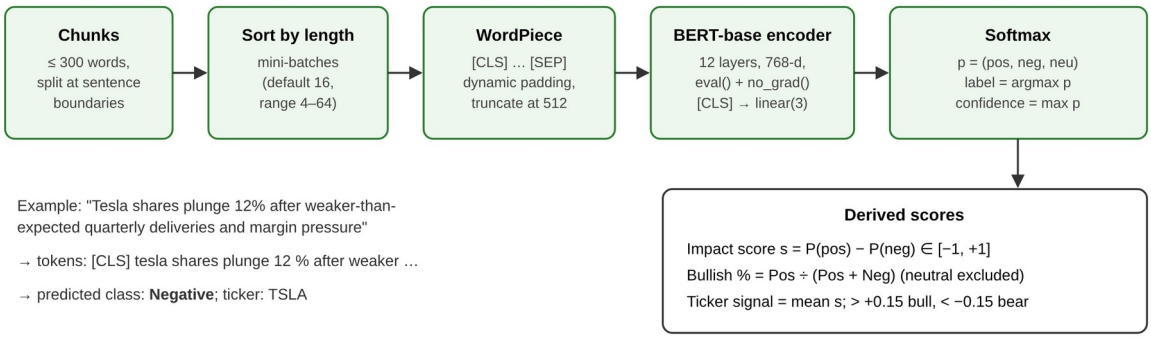


Figure 3.3: FinBERT inference procedure and derived scores

3.1.7 Ticker and entity extraction

Tickers are found with regular expressions and curated dictionaries, without any external API. The watchlist contains 68 symbols, covering US large caps (AAPL, MSFT, NVDA, TSLA, AMZN, GOOGL, META), Indian large caps (RELIANCE, TCS, INFY, HDFCBANK, ICICIBANK, SBIN, ITC, TATAMOTORS and others) and indices (NIFTY, BANKNIFTY, SENSEX, NASDAQ, DOW, SPX, FTSE, NIKKEI, DAX). A further 72 company names are mapped to tickers, for example “Infosys” → INFY and “Bank Nifty” → BANKNIFTY. Four rules are applied: cashtags such as $TSLA; exchange

prefixes such as NSE:TCS; capitalised words, accepted only if they are on the watchlist so that CEO, GDP and RBI are not counted; and company names. Users can add further tickers from the sidebar.

3.1.8 Market impact metrics and dashboard

In this project "market impact" means how strongly bullish or bearish the coverage of a stock or of the market is. It is a measure of the tone of the news, not of a price reaction. The metrics are defined in Table 3.1.

Table 3.1: Definitions of sentiment and market impact metrics

Metric	Level	Definition
Confidence	Article	Softmax probability of the predicted class (0 to 1)
Impact score	Article	P(positive) − P(negative), from −1 (very bearish) to +1 (very bullish)
Bullish %	Day	Positive ÷ (Positive + Negative); neutral articles are excluded because factual reporting has no direction
Net score	Day	Mean impact score over all articles
Ticker signal	Ticker	Mean impact score of the articles that mention the ticker; above +0.15 Bullish, below −0.15 Bearish, otherwise Neutral

The Streamlit dashboard displays a market-mood card with Bullish %, net score and a proportional positive/negative/neutral bar; KPI cards for article count, mean confidence, bullish-versus-bearish counts and the number of tickers; a Plotly donut chart of the sentiment mix; a stacked bar chart of sentiment by ticker with a top-N slider; a ticker leaderboard; filterable tables of the most positive and most negative articles; a "Show detected layout" overlay that draws a numbered box around every detected article on the page image; and a button that exports all results to finbert_daily_sentiment.csv. Predictions are kept in st.session_state so that moving a filter re-draws the charts without re-running the model. A separate "Test a single headline" mode shows the class probabilities for one sentence.

3.2 System Requirements

The application runs entirely on a CPU; a GPU is optional. Table 3.2 lists the recommended hardware and Table 3.3 lists the software that was installed and tested.

Table 3.2: Hardware requirements

Component	Minimum	Recommended
Processor	64-bit dual-core CPU	Quad-core CPU or better

Component	Minimum	Recommended
Memory (RAM)	4 GB	8 GB or more (FinBERT ≈ 110 M parameters plus OCR models)
Storage	2 GB free (Python packages and model weights)	5 GB free
GPU	Not required	CUDA GPU optional, speeds up OCR and inference
Network	Internet on first run to download FinBERT weights	—

Table 3.3: Software requirements and tested versions

Software / library	Version tested	Purpose
Python	3.13	Programming language
Streamlit	1.65	Web user interface and caching
Hugging Face Transformers	5.19	FinBERT model and tokenizer
PyTorch (CPU)	2.14	Neural network inference
PyMuPDF	1.28	PDF scanning, text blocks and page rendering
RapidOCR + ONNX Runtime	3.10 / 1.31	OCR with PP-OCRv5 models
pdfplumber / PyPDF2	0.11 / 3.0+	Fallback plain-text PDF extraction
pandas	3.0	Tabular results and aggregation
Plotly	7.1	Interactive charts
Operating system	Windows / Linux / macOS	Any OS supported by Python; Dev Container provided

3.3 Details of Data / Datasets

The system does not use live news feeds or market-data APIs; it analyses whatever file the user uploads. No model training was performed in this project: the pre-trained and fine-tuned ProsusAI/finbert checkpoint was used directly. The data sources used and supported are listed in Table 3.4.

Table 3.4: Input formats and test data

Data	Format	Description and handling
Financial Express, Delhi, 12 Oct 2026	PDF, 42 pages	Real e-paper, 1500 × 2384 pt pages, about 7–8 columns; text layer scrambled on all pages, so read with OCR
sample_news.txt	TXT, 17 lines	Hand-written Indian and US market headlines included in the project; one headline per line
Synthetic test newspaper	PDF, 2 pages	Generated 3-column paper with 4 known stories, header, footer, caption and stock table; used to verify the layout code
User CSV datasets	CSV	One article per row; text column detected automatically
Single headline	Text box	Typed by the user for instant testing

Data	Format	Description and handling
Financial PhraseBank [4]	Sentences	Not used directly; the dataset on which FinBERT was fine-tuned [1]

Output data consists of one record per article chunk with the fields text, title, sentiment, confidence, impact score, the three class probabilities, tickers and page number, which can be exported as CSV.

CHAPTER 4
IMPLEMENTATION DETAILS AND RESULT ANALYSIS

The system was implemented as a single Streamlit application, app.py (about 2,240 lines), organised into sections for configuration, model loading and batched inference, text-file and CSV parsing, PDF layout analysis and OCR, ticker extraction, dashboard components and the main application flow. The key parameters of the implementation are summarised in Table 4.1. All results below come from actual runs of the application or its test scripts on 12–13 October 2026.

Table 4.1: Key implementation parameters

Parameter	Value	Reason
Model	ProsusAI/finbert	BERT-base fine-tuned for 3-class financial sentiment
Maximum tokens	512	BERT input limit
Maximum words per chunk	300	Keeps chunks inside 512 tokens
Batch size	16 (range 4–64)	Balance of speed and memory on CPU
OCR resolution	110 dpi	Body text ≈ 21 px tall, enough for OCR
OCR minimum line score	0.5	Discards unreliable OCR lines
OCR recogniser	English PP-OCRv5, batch 32	Faster and keeps word spacing
Headline threshold	1.28 × body height	Between pull quotes (≤ 1.23) and headlines (≥ 1.31)
Column tolerance	15 pt	Groups text blocks into columns
Running-header margin	Top/bottom 7% of page	Repeated margin text is removed
Ticker signal band	±0.15	Separates bullish, neutral and bearish tickers

4.1 Results / Observations

4.1.1 Real newspaper: Financial Express, Delhi, 12 October 2026

The fast scan correctly identified all 42 pages as having a scrambled text layer, in 0.3 s, and routed them to OCR. Table 4.2 summarises the measured results.

Table 4.2: Results on the Financial Express e-paper

Measurement	Result
Pages with a scrambled text layer	42 / 42, detected in 0.3 s
OCR time per page (CPU, 110 dpi)	17.9 s (page 1), 23.0 s (page 3), 26.2 s (page 10)
End-to-end run, pages 1–2	OCR 38 s; 104 text regions → 30 chunks; 23 headlines
FinBERT inference on 30 chunks	6 s

Measurement	Result
Dashboard for pages 1–2	71% Bullish; 10 positive / 4 negative / 16 neutral; 81% mean confidence; 7 tickers
Pages 1, 3 and 10 combined	127 text regions, 27 headlines; layout analysis 0.06 s once OCR is cached

Figure 4.1 shows the detected layout of page 3, drawn by the application's “Show detected layout” option with one numbered box per article. The stories “Rice exports may take a hit next year”, “NTPC seeking role beyond generation” and “Registrations of EVs jump 84%; UP leads among states” were each recovered with their full titles and their own body text, even though their headlines span different numbers of columns. On page 10 (the Opinion page), stories such as “Barricading Young India”, “Jio changed telecom, Starlink faces the test” and “The employee who does just enough” were also separated correctly. Figure 4.2 shows pages 1 and 10.



Figure 4.1: Detected article layout on page 3 of the Financial Express e-paper



Figure 4.2: Detected article layout on page 1 (left) and page 10 (right)

4.1.2 OCR performance and tuning

OCR is the slowest stage. With the default recogniser the densest page (page 10) took 102 s. Switching to the English PP-OCRv5 recogniser with batches of 32 lines reduced this to 31 s, and on a test crop it reduced the number of words wrongly run together from 17 to 7 (for example “offthenational capitalin” became “off the national capital in”). Figure 4.3 shows the per-page times with the final settings and the effect of the tuning.

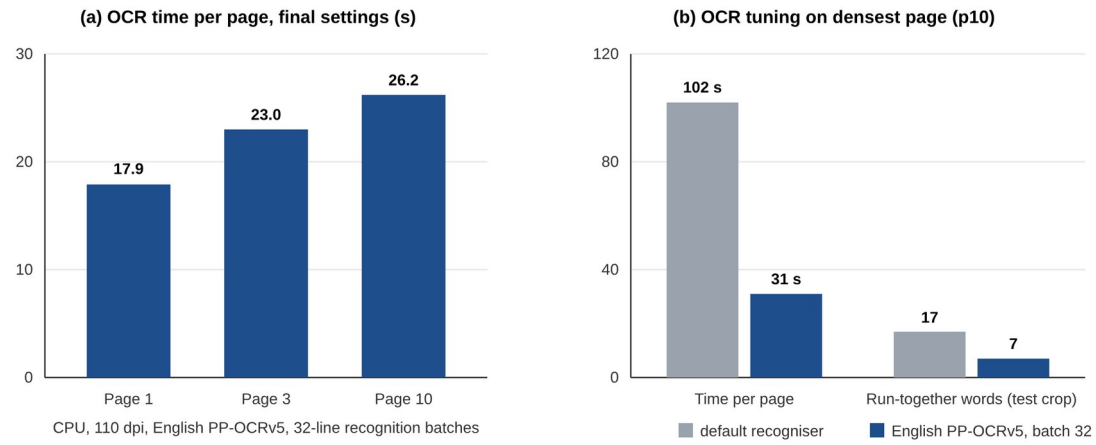


Figure 4.3: OCR time per page and the effect of OCR tuning

4.1.3 Synthetic three-column test paper

On the generated two-page, three-column test newspaper with four known stories, both the PyMuPDF block path (for readable PDFs) and the OCR path recovered every story with no text mixed between stories. The running header, footer, image caption and stock-price table were all removed. This confirms that the layout code is correct on a page whose ground truth is known exactly.

4.1.4 Sample headline set

The 17 headlines in sample_news.txt were classified with a mean confidence of 93%: 7 positive, 8 negative and 2 neutral, giving a market mood of 53% Bearish and a net score of −0.09. In single-headline mode, “Tesla shares plunge 12% after weaker-than-expected quarterly deliveries” was classified as Negative. Figure 4.4 compares the sentiment mix of the two runs, and Table 4.3 shows the tickers extracted from each sample headline.

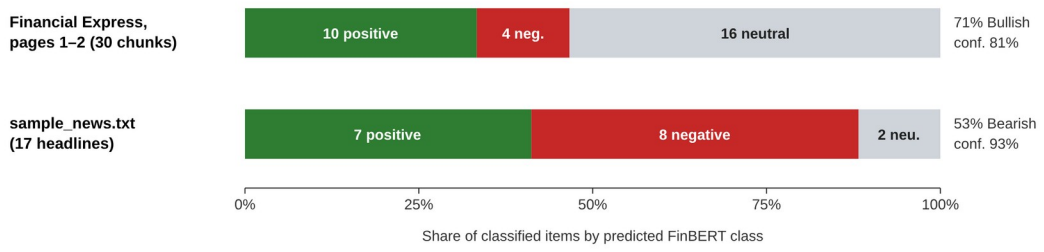


Figure 4.4: Sentiment distribution for the newspaper run and the sample headline set

Table 4.3: Ticker extraction on sample_news.txt

#	Headline (shortened)	Tickers	Rule
1	Tesla shares plunge 12% after weaker-than-expected …	TSLA	Company name
2	Apple ($AAPL) reports record iPhone revenue …	AAPL	Cashtag / name
3	Nifty and Sensex hit fresh all-time highs …	NIFTY, SENSEX	Index names
4	Reliance Industries posts 18% jump in net profit …	RELIANCE	Company name
5	Infosys cuts full-year revenue guidance …	INFY	Company name
6	HDFC Bank shares slide after … bad loans	HDFCBANK	Company name
7	NSE:TCS wins a $2 billion multi-year deal …	TCS	Exchange prefix
8	Nvidia market value surpasses $4 trillion …	NVDA	Company name
9	Microsoft to lay off 6,000 employees …	MSFT	Company name
10	The Reserve Bank of India will announce … policy	—	None (correct)
11	Crude oil prices fall 4% … dragging ONGC lower	ONGC	Watchlist symbol
12	Amazon stock rallies after cloud unit AWS …	AMZN	Company name
13	Tata Motors recalls 40,000 vehicles …	TATAMOTORS	Company name
14	Bank Nifty ends flat as investors await …	BANKNIFTY	Company name
15	Zomato turns profitable for the third … quarter	ZOMATO	Company name
16	Meta faces a fresh $1.3 billion EU fine …	META	Company name
17	Wipro shares remain unchanged after … CFO	WIPRO	Company name

All 17 distinct tickers referred to in the file were found. Headline 3 correctly produced two indices, and in headline 14 “Bank Nifty” was mapped to BANKNIFTY rather than NIFTY. Headline 10, about the Reserve Bank of India, correctly produced no ticker: “RBI” is not on the watchlist and is not mistaken for a stock.

4.2 Analysis of the Results

Effectiveness of the extraction stage. The most important observation is that, without the extraction stage, the sentiment model could not have been applied to the real newspaper at all: every standard PDF library returned unreadable characters. The garbled-page test identified this situation automatically and cheaply. The combination of headline detection, headline ownership and XY-cut then recovered complete stories with correct titles on pages with seven to eight columns and with headlines spanning different numbers of columns. Calibrating the headline threshold on measured values (1.23 for pull quotes versus 1.31 for headlines) was essential; a threshold taken from typography rules alone would have split stories at pull quotes.

Computational cost. The cost is dominated by OCR. With 18–26 s per page on a CPU, a full 42-page paper is estimated to take about 15–20 minutes, whereas FinBERT classified

30 chunks in 6 s and layout analysis took only 0.06 s once OCR results were cached. Length-sorted batching, dynamic padding and no_grad inference keep the language-model stage fast enough that it is not the bottleneck. For this reason OCR results are cached per page and the application reads pages 1–8 by default.

Sentiment output. The mean confidence was 81% on newspaper chunks and 93% on clean headlines. The lower value for the newspaper is expected: OCR text contains occasional errors, chunks are longer and often mix several statements, and FinBERT was fine-tuned on short, clean sentences. A large share of the newspaper chunks (16 of 30) were classified as neutral; inspection showed that these include factual reports as well as bank legal notices and advertisements that reach the model. Because Bullish % excludes neutral items, such noise dilutes the counts but does not bias the bullish-versus-bearish ratio. The sample set, which contains mostly clearly positive or negative events, was classified with only two neutral headlines, which matches its design.

Entity linking. The rule-based extractor achieved complete recall on the sample file without false positives, and the watchlist rule prevented common capitalised abbreviations from being counted as tickers. Its limitation is coverage: symbols outside the watchlist are found only if the user adds them.

Validity. The labels have not yet been compared with human-annotated ground truth for this newspaper, so no accuracy figure is claimed for this data. The measured quantity is the tone of the coverage; no price data or event study was used, so the dashboard does not claim to predict returns.

4.3 Comparison with Existing Work

Table 4.4 compares the system with representative existing work along the dimensions that matter for practical use. The classification accuracy of the system is inherited from FinBERT, so the contribution of this project lies in the stages around the model: robust input handling, layout analysis, entity linking and aggregation.

Table 4.4: Comparison of the proposed system with existing work

Feature	Lexicon (LM [5], VADER [8])	FinBERT [1] as published	This project
Sentiment method	Word lists / rules	Fine-tuned BERT	Fine-tuned BERT (ProsusAI/finbert)
Context awareness	No	Yes	Yes

Feature	Lexicon (LM [5], VADER [8])	FinBERT [1] as published	This project
Reported accuracy	Lower than learned models [1], [4]	0.86 (0.97 all-agree)	Inherited; not re-measured on newspaper
Input granularity	Documents / posts	Single sentences	Whole newspaper, TXT, CSV, headline
Multi-column layout	Not handled	Not handled	XY-cut + headline ownership
Scrambled / scanned PDF	Not handled	Not handled	Auto-detection + PP-OCRv5
Long text	Any length	Truncated at 512 tokens	Sentence-aware 300-word chunks
Ticker linking	No	No	68 symbols + 72 names, 4 rules
Daily aggregation	Manual	No	Bullish %, net score, ticker signal
User interface	Library	Library / model	Interactive dashboard + CSV export

Compared with lexicon methods, the system understands context and financial phrasing. Compared with FinBERT used on its own, it can be applied to an entire real newspaper rather than to pre-separated sentences, it does not lose text to truncation, and it turns per-article labels into per-ticker and per-day indicators. Compared with studies such as Tetlock [6], it does not yet relate the sentiment to subsequent market returns, which is the main item of future work.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project developed an end-to-end system that reads a complete daily financial newspaper and produces an interactive summary of its sentiment. The system meets its objectives: it accepts PDF, TXT, CSV and single-headline input; it detects scrambled PDF text layers automatically and reads such pages with OCR; it reconstructs individual articles from multi-column pages using calibrated headline detection, a headline-ownership rule and a recursive XY-cut; it classifies each article with FinBERT using efficient, truncation-free batched inference; it links articles to stocks and indices; and it presents clear daily and per-ticker metrics in a Streamlit dashboard with CSV export.

On a real 42-page e-paper whose text layer could not be read by any standard PDF library, the system recovered complete stories with correct titles and reported a 71% bullish mood with 81% mean confidence for pages 1–2. OCR tuning reduced processing time on the densest page by a factor of more than three. On a sample headline set the system achieved 93% mean confidence and detected every referenced ticker. The project shows that the main difficulty in applying a state-of-the-art language model to real financial documents lies in getting clean, correctly segmented text to the model, and that classical document-analysis algorithms combined with modern OCR solve this problem effectively.

The main limitations are that “market impact” is measured as the tone of the coverage rather than as a price reaction, that the labels have not been evaluated against human annotations, that OCR is slow on a CPU (about 15–20 minutes for a full paper), that the layout heuristics were tuned on one newspaper, that stories continued on another page are treated as separate pieces, that sentiment is assigned per article rather than per entity, and that only English text is supported.

5.2 Suggestions for Future Directions

1. Measure real market impact: retrieve daily prices from an exchange data API and run an event study that compares each ticker's sentiment with its return over the next one and five trading days, using NIFTY or SENSEX as the benchmark.

2. Evaluate accuracy: hand-label 100–200 newspaper articles and report accuracy and F1 for FinBERT on Indian financial news; if needed, fine-tune the model on this data.
3. Entity-level sentiment: score only the sentences that mention each ticker, so that an article that is positive for one company and negative for another is not averaged.
4. Filter advertisements and legal notices before classification with a small classifier or keyword rules.
5. Faster OCR: use a GPU, process pages in parallel, or OCR only the pages the user selects.
6. History and trends: store each day's results in a database and plot the market mood and ticker signals over weeks and months.
7. Generalise the layout analysis to other newspapers, join stories that continue on later pages, and add support for Hindi and other Indian languages with multilingual models.

REFERENCES

Journal / Conference Papers

[1] D. Araci, “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models”, arXiv preprint arXiv:1908.10063, 2019.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), Minneapolis, USA, pp. 4171–4186, June 2019.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention Is All You Need”, Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS), Long Beach, USA, pp. 5998–6008, December 2017.

[4] P. Malo, A. Sinha, P. Korhonen, J. Wallenius and P. Takala, “Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts”, Journal of the Association for Information Science and Technology, vol. 65, no. 4, pp. 782–796, 2014.

[5] T. Loughran and B. McDonald, “When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks”, The Journal of Finance, vol. 66, no. 1, pp. 35–65, 2011.

[6] P. C. Tetlock, “Giving Content to Investor Sentiment: The Role of Media in the Stock Market”, The Journal of Finance, vol. 62, no. 3, pp. 1139–1168, 2007.

[7] J. Bollen, H. Mao and X. Zeng, “Twitter Mood Predicts the Stock Market”, Journal of Computational Science, vol. 2, no. 1, pp. 1–8, 2011.

[8] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text”, Proceedings of the International AAAI Conference on Weblogs and Social Media (ICWSM), Ann Arbor, USA, pp. 216–225, June 2014.

[9] Y. Kim, “Convolutional Neural Networks for Sentence Classification”, Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, pp. 1746–1751, October 2014.

[10] A. H. Huang, H. Wang and Y. Yang, “FinBERT: A Large Language Model for Extracting Information from Financial Text”, Contemporary Accounting Research, vol. 40, no. 2, pp. 806–841, 2023.

[11] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg and G. Mann, “BloombergGPT: A Large Language Model for Finance”, arXiv preprint arXiv:2303.17564, 2023.

[12] G. Nagy and S. Seth, “Hierarchical Representation of Optically Scanned Documents”, Proceedings of the International Conference on Pattern Recognition (ICPR), Montreal, Canada, pp. 347–349, 1984.

[13] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang and H. Wang, “PP-OCR: A Practical Ultra Lightweight OCR System”, arXiv preprint arXiv:2009.09941, 2020.

[14] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing”, Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), pp. 38–45, November 2020.

[15] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory”, Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[16] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi et al., “Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation”, arXiv preprint arXiv:1609.08144, 2016.

Reference / Hand Books

[17] D. Jurafsky and J. H. Martin, “Speech and Language Processing”, Pearson Prentice Hall, 2nd Edition, 2009, ISBN 978-0131873216.

[18] I. Goodfellow, Y. Bengio and A. Courville, “Deep Learning”, MIT Press, 1st Edition, 2016, ISBN 978-0262035613.

[19] S. Russell and P. Norvig, “Artificial Intelligence: A Modern Approach”, Pearson, 4th Edition, 2020, ISBN 978-0134610993.

Web

[20] ProsusAI/finbert model card, Hugging Face, https://huggingface.co/ProsusAI/finbert

[21] Streamlit documentation, Streamlit Inc., https://docs.streamlit.io

[22] PyMuPDF documentation, Artifex Software, https://pymupdf.readthedocs.io

[23] RapidOCR, RapidAI (GitHub), https://github.com/RapidAI/RapidOCR

[24] Plotly Python graphing library, Plotly, https://plotly.com/python/

[25] Project source code: Financial Sentiment & Market Impact Dashboard, GitHub, https://github.com/TejasLall/FinBERT